UX suggestions for OpenAI

Auelius

Independent product feedback

2 November 2026

Abstract

Product feedback proposals drawn from sustained use of ChatGPT, Codex and related assistant workflows. The central opportunity is to reduce the user's coordination burden while making context, permissions, persistence and completion easier to trust.

This sanitized review preserves fifteen UX suggestions and their evidence limits. Examples are generalized; identifying details, private links and operational records are omitted. These are historical observations and proposals, not a comprehensive audit of current product capabilities or feedback already submitted.

Contents

UX-001 A user-governed assistant feedback channel . . . 2
UX-002 Reliable editing of persistent working notes . . . 2
UX-003 Compact overviews without compulsory interaction . . . 3
UX-004 Distinguish saved content from verified rendering . . . 3
UX-005 Separate identity, context and capabilities . . . 3
UX-006 Reduce conversational effort without reducing reasoning quality . . . 4
UX-007 Recoverable answers and transparent waiting states . . . 4
UX-008 Canonical history with explicit completeness . . . 5
UX-009 An automatic active context workspace . . . 5
UX-010 Orchestration without manual message relaying . . . 7
UX-011 Verified history rather than plausible reconstruction . . . 7
UX-012 Reliable documentation access from the embedded browser . . . 8
UX-013 Observable screen sharing reception . . . 8
UX-014 Delegate from Dot into normal Chat for knowledge development . . . 8
UX-015 Supported Programmatic Automation for ChatGPT and Codex . . . 9

UX-001 A user-governed assistant feedback channel

Observation

An assistant identified friction in its own workflow but misunderstood a suggestion for provider feedback as a request to build another local tool. The available workflow did not provide a direct assistant-originated feedback route.

Proposal

Let assistants draft concise product feedback for user review, with visible content, provenance, consent controls and traceable submission status. Feedback routing must remain distinct from silently forwarding conversation data.

Benefit

Observations from sustained use could inform improvements without requiring the user to coordinate every report. This is a proposed capability, not a claim that every existing feedback route has been tested.

Further observation and proposal

A participant asked for an assistant to submit explicitly approved feedback and notify them when improvements addressed it. The request does not establish submission. Retain a receipt and, with consent, connect later changes to the original issue. Distinguish feedback that has been drafted, submitted or acknowledged from improvements that have been announced, made available to the account or verified through testing.

UX-002 Reliable editing of persistent working notes

Observation

The source records unnecessary handoffs for edits the assistant could perform through native file access. A user reported repeated handoffs before a direct write succeeded. Other exchanges show the user relaying completion evidence between assistants. Acknowledging a correction did not consistently prevent a later unnecessary handoff. A direct comparison of two same-purpose files found divergent updates. This established split content, not why the copies existed or whether duplication was intentional. A later full artifact read also established that a previously questioned entry had been saved. Status records should be corrected when they no longer reflect the verified outcome.

Proposal

Make native write capability discoverable before delegation. Retain the selected artifact identity across conversations, surface same-title variants and divergent edits before saving, and provide a verified persistence receipt. Expose authorized handoff receipts and shared-artifact references so users need not relay screenshots or status messages.

Evidence limits

Some handoff counts and search outcomes were participant reports, not replayed operations. The observed cases support capability-first routing and honest saved/pending status, not a universal claim of cross-assistant invisibility.

Benefit

Persistent notes become reliable working material rather than another coordination task.

Further observation and proposal

After requesting an experiment-record update, a participant had to ask where it was stored. They later required source-discussion links in follow-up records and praised being able to return to the

reasoning. These are user statements, not independent inspection of record creation or a storage backend. Consequential saved/added claims should name the actual persistent destination and provide a stable link, distinguishing conversation-only capture from artifact or database updates.

UX-003 Compact overviews without compulsory interaction

Observation

A user requested a faster recurring briefing that preserved its information and categories while reducing clicks and duplicated content. The editing report described a compact redesign with preserved links and removed duplicates; the user accepted it provisionally. The editor's reported preservation was not independently visually audited.

Proposal

Provide a complete compact overview with optional detail on demand. Avoid compulsory category or story selection and multiple renditions of the same material.

Benefit

A short briefing should save attention rather than turn into an interface-navigation task.

UX-004 Distinguish saved content from verified rendering

Observation

Native Page reads and writes succeeded, while the editing assistant reported that it could not inspect the final embedded presentation. Successful persistence did not establish successful rendering. The evidence does not show that the Page was broken or that preview access is universally unavailable.

Proposal

Provide a lightweight authorized preview or equivalent inspection route. Distinguish saved, structurally checked and visually verified results, and state the remaining uncertainty when preview access is absent.

Benefit

Users should not have to discover presentation defects merely because an assistant can save an artifact but cannot inspect it.

UX-005 Separate identity, context and capabilities

Observation

User feedback distinguishes a continuing assistant identity from its execution environment, and distinguishes that assistant from separately delegated workers with different memories and permissions. Reports also describe confusion when similarly labelled actions route to environments with different resource access, or similarly named local and cloud projects refer to different resources. These are bounded reports, not a verified universal client-routing rule.

Proposal

Show the acting identity, execution host, resource identity, available capabilities, tested availability and authorized scope at the point of action. Check required resources before dispatch and make an environment change explicit. Continuity must not silently transfer authority. Offer an explicit protected-context mode with a persistent indicator and a change trail. Enforce protected-resource rules independently of conversational classification and across delegated workers. Permission to discuss a topic is not permission to modify its instructions or resources.

Evidence limits

Protected-context enforcement is a design proposal, not an implemented guarantee; the evidence does not demonstrate a bypass. A failed resource listing in one task does not establish a product-wide limitation; a completed turn does not prove its requested result was obtained.

Application modes

A participant reported difficulty identifying the active ChatGPT or Codex mode when the sidebar was closed, while similar conversation interfaces exposed different capabilities. Their feedback distinguished those major modes from subordinate Chat/Work workflows. Keep the major mode and execution environment visible near the composer independently of the sidebar. This is a bounded interface report, not an independently reproduced current limitation.

Extensibility proposal

The participant also explicitly asked for custom major modes, comparing them to extensible editor modes. Support user-defined major modes with visible scope, capabilities and permitted actions; those safeguards are a design synthesis, not a tested implementation or a verbatim user specification.

Instruction scope

An assistant attributed a separate lookup's restriction to the rules of a read-only recurring review; the user clarified the narrower scope. Show the source and scope of operative instructions. Enforce the review's boundary without silently turning it into a global prohibition on separately authorized work. Distinguish one-task authorization from a change to a standing rule and preserve consequential-action safeguards.

UX-006 Reduce conversational effort without reducing reasoning quality

Observation

The user's feedback criticizes elaborate restatements that add little beyond the user's argument and unnecessary closing questions that create more work. The user asks for compact examination while welcoming depth that adds substance. The assistant acknowledged this preference; sustained improvement was not tested.

Proposal

Evaluate responses for useful evidence, distinctions, conclusions and necessary clarification rather than length or continued engagement. Honor concise delivery independently of reasoning effort. Do not end every response with an unsolicited question.

Benefit

Careful reasoning should save the user's attention rather than consume it through rhetorical expansion.

UX-007 Recoverable answers and transparent waiting states

Observation

A screenshot showed completed-work indicators without the expected answer visible. A later reply appeared. This demonstrates a mismatch between completion signalling and visible output, not whether the answer was lost, delayed, collapsed or outside the viewport. A separate user report described an answer disappearing and later returning. Other exchanges show the user asking whether work was still progressing, followed by an assistant report of a stalled query and bounded recovery. These are conversation records, not continuous runtime telemetry or a verified technical cause.

Proposal

Make a completed result recoverable, distinguish delivery acknowledgement from finished computation, and provide restore or retry controls for missing output. Show the last confirmed step, current operation and freshness, with separate running, timed-out, waiting-for-input and completed-with-deliverable states. Before retrying an uncertain operation, reconcile its outcome through a bounded read-only check. Never blindly replay a write after a timeout. Confirm that documentation was saved persistently separately from having discussed an issue.

Benefit

Users should not have to prompt merely to learn whether work continues or whether its answer was delivered.

UX-008 Canonical history with explicit completeness

Observation

User feedback asks for reliable preserved discussions without losing conclusions or confusing original exchanges with later summaries and interpretations. The retrieved source material omitted messages or attachments. A relayed migration audit distinguished sampled content, known membership and unresolved message references; its totals were a historical report, not a current account census. An agent report described a migration batch recovering from rate-related errors with bounded operations, saved-state verification and permitted retries. It is an inspected report, not an independent replay of native operations. Broader migration remained incomplete at that checkpoint.

Proposal

Provide a provider-maintained, user-controlled record searchable by the user and authorized assistants. Preserve messages, actual timestamps, participant/source identities and attachments, disclose gaps and verify completeness before describing a record as canonical or safe to replace originals. Keep summaries and evolving interpretations separate and linked to evidence. Expose a paginated inventory with current membership and recoverable content. Keep proposed classification separate from actual membership and migration authorization. Make bulk organization resumable with per-item receipts, actionable cooldown information and read-only reconciliation before retries.

Safeguards

Preserve original message dates separately from move/update timestamps. Moving a chat is neither new discussion nor proof of a complete backup. Conditional interest in an archive does not authorize deleting originals.

Benefit

Users can revisit intellectual work without maintaining large collections of chat containers by hand.

Source continuity

A participant explicitly requested source-discussion links in derived idea records and later reported that the return link was useful. Preserve those links alongside the derived record so the original reasoning stays recoverable; neither a summary nor a task replaces the source conversation. The observed benefit is user-reported, not an independent storage audit.

UX-009 An automatic active context workspace

Observation

A screenshot and user reports describe manually naming and revisiting several active workstreams. Other exchanges show the assistant reconstructing decisions and next steps after an interruption. These

examples establish navigation and resumption friction, not lost stored data or complete visibility into all tasks.

Proposal

Discover and group ongoing contexts across conversations. Support direct switching, relationships between contexts without forced merging, resumable decisions and activities, pending-input indicators and user-correctable lifecycle states. Contexts can span chats, agents, files and decisions; they should not be reduced to titles or mandatory manual containers. Provide a concise resumption view with the last accepted decision, unfinished work, next permitted step and evidence gaps. Organize around objectives and scoped workers, reusing specialist context when useful without requiring a persona for every task.

Safeguards

Automatic classification must be inspectable and correctable and must not transfer permissions. A resumption view does not authorize starting work. Keep inspection, simulation and live execution distinct. Discussion volume or an interesting proposal does not establish implementation priority.

Benefit

Users manage the work rather than the containers needed to remember it.

Mode preferences and navigation

A participant distinguished an available filter control from a persistent mode-specific default. A separate exchange records a failed native link and the assistant's acknowledgment that it had used another product's identifier; the user also reported a shortcut failing despite a successful discussion move. Persist selected per-mode filters with an explicit combined view, and resolve native links from typed resource identifiers and the current client. A valid URI or registered shortcut is not proof it works in that client.

Direct mode switching

The participant specifically requested a shortcut between ChatGPT and Codex major modes, distinct from a Chat/Work toggle or a conversation filter. Provide a discoverable direct switch command with a documented, user-configurable shortcut and clear destination-mode feedback. This is a requested interaction, not a verified claim that current clients lack such a shortcut.

Lifecycle and cross-device use

Historical feedback describes wanting to retire a completed project while preserving its material for later comparison, but not finding a project-level archive action. Another exchange requests predictable chronology, labels and filtering, then identifies a desktop-only workaround as insufficient for mobile navigation. Provide reversible project-level active/dormant/archived states, searchable preserved content and shared organization state across devices. Distinguish last-message ordering from metadata-change ordering. These reports do not establish the current sorting algorithm or universal absence of those capabilities.

Cloud-only organization

A participant asked for automatic classification and correction of misplaced discussions without another desktop job. The assistant described a successful small-scale test as desktop-native, which did not demonstrate cloud-only sorting. Make actual membership and environment-specific organization capabilities discoverable, with inspectable, user-correctable classification. Do not require desktop mediation where an authorized cloud route is available.

UX-010 Orchestration without manual message relaying

Observation

A user relayed review instructions between assistants and managed a waiting checkpoint. Historical exchanges show a request for transparent execution routing, followed by an assistant proposing extra coordination before checking an available direct route. A later user report said direct exchange worked without manual relaying. That is a reported success, not an independently replayed integration. Other participant reports describe handoff failures, permitted retries and recovered work. One rejected dispatch was followed by the user identifying their own error; it should not be attributed to a product defect.

Proposal

Transfer context, constraints and intended outcomes to the appropriate authorized worker. Track ownership, acknowledgement, progress and verified results, and update blocked statuses when handoffs are completed. Expose distinct authorization-denied, capacity-unavailable, dispatch-rejected, delivered and execution-started states. Preserve prepared work across permitted retries without duplicate workers or lost changes. Expose each recurring operation's identity, owning scheduler, destination, current configuration and verified pause acknowledgement. Updating one surface does not prove a duplicate routine elsewhere stopped. Distinguish requested, changed and verified states.

Safeguards

Coordination is not authorization. Respect agent scope, resource ownership, protected contexts and controls on conflicting edits. Consequential actions retain their approval requirements. Verify the relevant outcome before advancing dependencies; a promise or finished turn is insufficient. Evaluate end-to-end completion across discussion, capability discovery, handoff and persistence. Preserve useful exploratory conversation while removing repeated setup and execution repair. Participant assessments of friction are not measured failure rates.

Benefit

Users should not have to act as the message relay, scheduler and status reconciler for capable assistants.

Integrated transfer and retained judgment

User feedback explicitly rejects compulsory copy-paste as an integration step. It also distinguishes unnecessary delegation of questions needing longitudinal context from useful delegation when another worker has required resource access. Route by both access and retained context, transfer authorized material directly, and leave copy-paste as an optional convenience. This does not imply a ban on useful delegation or prove universal context loss.

UX-011 Verified history rather than plausible reconstruction

Observation

In a deliberate continuity test, a user introduced an invented additional option as though it belonged to an earlier agreement. The assistant confidently accepted it, then acknowledged that the historical claim had not been verified. Subsequent promises to improve were not accompanied by a verifiable configuration change. The invented option was not an actual user activity. This is one observed error, not a measured general failure rate.

Proposal

When consequential prior agreements are invoked, distinguish retrieved conversation evidence, memory-derived claims, inference and unverified assertions. Retrieve evidence before confidently asserting continuity. Keep ordinary conversation natural, using provenance labels when they affect interpretation.

Distinguish behavioral intentions, remembered feedback, configured preferences and verified execution changes in user-visible status.

Benefit

Longitudinal trust depends on knowing whether an assistant is remembering an agreement or constructing a plausible story.

UX-012 Reliable documentation access from the embedded browser

Observation

An inspected screenshot showed the OpenAI Help Center blocked by a human-verification challenge inside Codex. The user reported that repeated attempts were rejected. The screenshot establishes the challenge; repeated rejection is a user report. The cause was not verified, and an interface connection indicator did not explain it.

Proposal

Make official documentation reliably accessible from the working environment. Support functioning verification or a seamless fallback to the system browser that preserves the original destination and relevant task context. Explain failed access and usable next steps instead of encouraging repeated attempts at the same rejected challenge.

Benefit

Consulting product documentation should not interrupt work or require reconstructing the destination and context.

UX-013 Observable screen sharing reception

Observation

In a historical screen-sharing test, a participant could not establish whether the assistant was receiving the shared visuals. The assistant described benefits of screen sharing despite reporting no visual access, then corrected its misleading explanation. An uploaded image provided a reported alternative, after which the user accepted the assistant's reported limitation.

Proposal

Expose the selected visual input, last received frame and freshness, and receiving/unavailable states to both user and assistant. Offer a minimal visual-reception test and a clear fallback. Distinguish newly perceived screen content from remembered chat content.

Evidence limits

The text exchange was read, but audiovisual payloads were omitted and neither delivery nor image interpretation was independently replayed. This supports clearer capability explanations, not a verified technical cause or current product-wide limitation.

Benefit

Users can tell whether screen sharing is supplying useful input without repeatedly testing or correcting the assistant.

UX-014 Delegate from Dot into normal Chat for knowledge development

Observation

A user requested a separate exploratory discussion, but the assistant used a discussion-only Work session as a workaround. The user distinguished this from normal Chat and clarified that delegation should

originate from Dot. The desired outcome was advances in understanding, resources and plans, not code.

Proposal

Let Dot explicitly choose normal Chat for cognitive exploration or Work for execution. Create a separate Chat discussion with relevant, authorized context and appropriate project placement. Show what context was transferred or omitted, and return a concise synthesis of learning, useful resources, plans and unresolved questions with source provenance.

Boundaries

Do not silently turn exploration into execution or assume a coding deliverable. Verify context transfer and project placement within existing permissions. Direct normal-Chat creation was not exposed in the observed session; this is not a product-wide capability claim or a guarantee of seamless context transfer.

Benefit and relationship

Cognitive delegation becomes a first-class workflow with outcomes appropriate to inquiry. It complements UX-005's mode clarity and UX-010's orchestration while distinguishing Dot-to-normal-Chat delegation from execution work.

UX-015 Supported Programmatic Automation for ChatGPT and Codex

Category and priority

UX / Developer Experience / Agent Integration. Priority: Medium–High.

Observation

A private micro-harness was developed solely to rename and organize the user's own ChatGPT discussions. No reverse engineering, safeguard circumvention or file exfiltration was undertaken; publication was deliberately rejected. The experiment suggested broader automation possibilities, not a claim of supported access.

Proposal

Provide an official permission-scoped `atlas.*` surface for capability discovery; chat search, rename and project moves; Codex task listing, inspection and delegation; and authorized context transfer. Include explicit authorization, granular permissions, audit trails, verifiable outcomes, documented rate limits and security boundaries.

Illustrative proposal only

These JavaScript calls are not deployed endpoints.

```
await atlas.openai.capabilities();
const chats = await atlas.chat.search({query:'Emacs analogy'});
await atlas.chat.rename(chats[0].id,'Codex and Emacs');
await atlas.chat.moveToProject(chats[0].id,'Atlas — Infrastructure');
const tasks = await atlas.codex.tasks.list({status:'running'});
await atlas.codex.tasks.inspect(tasks[0].id);
await atlas.codex.tasks.delegate({objective:'Review architecture',context:
  atlas.chat.current()});
```

Principle

Make supported automation easier and safer than building an unofficial harness.

Evidence and sharing scope

These suggestions combine directly read exchanges, inspected artifacts or screenshots, user reports, agent reports and proposed remedies. Each entry retains the distinctions that affect its claims. They are qualitative findings, not prevalence estimates, benchmark results or a claim of complete account coverage. Private provenance is intentionally excluded from this review. No linked private documents, personal quotations, identifying project details or operational identifiers are included. The complete private record remains separate. At the time this review was prepared, it had not been submitted externally.